

ECE391: Computer Systems Engineering
Machine Problem 3

Fall 2023

Checkpoint 1: 5:59pm Monday 16 October

Checkpoint 2: 5:59pm Monday 23 October

Checkpoint 3: 5:59pm Friday 3 November

Exam 2: 7:00 - 9:00 pm Monday 6 November

Checkpoint 4: 5:59pm Monday 13 November

Thanksgiving Fall Break

Checkpoint 5: 5:59pm Sunday 3 December

Final Demos: TBD

Illinux 391

Contents

1	Introduction	3
2	Using the Group Repository	3
3	Getting Started: Booting and GRUB	4
4	The Pieces	4
5	Testing	5
6	What to Hand in	5
6.1	Checkpoint 1: Processor Initialization	5
6.1.1	Group Repository	5
6.1.2	Load the GDT	5
6.1.3	Initialize the IDT	5
6.1.4	Initialize the Devices	6
6.1.5	Initialize Paging	6
6.1.6	Troubleshooting/Debugging	6
6.1.7	Test Coverage	6
6.1.8	Checkpoint 1 Handin	7
6.2	Checkpoint 2: Device Drivers	8
6.2.1	Create a Terminal Driver	8
6.2.2	Parse the Read-only File System	8
6.2.3	The Real-Time Clock Driver	8
6.2.4	Test Coverage	8

6.2.5	Checkpoint 2 Handin	9
6.3	Checkpoint 3: Starting System Calls and Tasks	10
6.3.1	Support System Calls	10
6.3.2	Tasks	10
6.3.3	Support A Loader	10
6.3.4	Executing User-level Code	10
6.3.5	Process Control Block	11
6.3.6	Test Coverage	11
6.4	Checkpoint 4: Completing System Calls and Tasks	11
6.4.1	Test Coverage	11
6.5	Checkpoint 5: Scheduling	13
6.5.1	Multiple Terminals and Active Tasks	13
6.5.2	Scheduling	13
6.5.3	Test Coverage	13
6.5.4	Checkpoint 5 Handin	14
6.6	Extra Credit	15
6.6.1	Signals	15
6.6.2	Dynamic Memory Allocation	15
6.6.3	Other Ideas	15
7	Grading	16
8	Appendix A: The File System	17
8.1	File System Utilities	17
8.2	File System Abstractions	18
9	Appendix B: The System Calls	19
10	Appendix C: Memory Map and Task Specification	21
11	Appendix D: System Calls, Exceptions, and Interrupts	22
12	Appendix E: Stack Switching and the TSS	22
13	Appendix F: Signals	23
14	Appendix G: Troubleshooting	25
14.1	Debugging with QEMU	25
14.2	/mnt/tmpmp3 Compile Error	25
14.3	Buffer I/O error on device loop0, logical block ##### lost page write due to I/O error on loop0	25

1 Introduction

Read the whole document before you begin, or you may miss points on some requirements (for example, the bug log).

In this machine problem, you will work in teams to develop the core of an operating system. We will provide you with code that boots you into protected mode, sets up the GDT, LDT, and initial TSS, and maps a read-only file system image into physical memory for you. You must set up the interrupt descriptor table (IDT), basic paging support for tasks, separate 4 MB pages for the kernel and applications, initialize a few devices, write the system call interface along with ten system calls, provide support for six tasks from program images in the file system which interface with the kernel via system calls, multiple terminals and basic scheduling.

The goal for the assignment is to provide you with hands-on experience in developing the software used to interface between devices and applications, i.e., operating systems. We have deliberately simplified many of the interfaces to reduce the level of effort necessary to complete the project, but we hope that you will leave the class with the skills necessary to extend the implementation that you develop here along whatever direction you choose by incrementally improving various aspects of your system.

2 Using the Group Repository

You should be receiving an invite to your group repository within the next two days or so. Your repo can be found at: <https://gitlab.engr.illinois.edu/ece391-fa21/mp3-group-XX> where XX is your assigned group number. Please check the course website Assignment page to know what is your group number.

To use Git on a lab computer, you'll have to use Git Bash. You are free to download other Git tools as you wish, but this documentation assumes you are using Git Bash. To launch Git Bash, click the Start button in Windows, type in `git bash`, then click on the search result that says Git Bash.

Run the following commands to make sure the line endings are set to LF (Unix style):

```
git config --global core.autocrlf input
git config --global core.eol lf
```

Switch the path in git-bash into your Z: drive by running the command: `cd /z.`

If you do NOT have a ssh-key configured, clone your git repo in Z: drive by running the command, it will ask for your NETID and AD password:

```
git clone https://gitlab.engr.illinois.edu/ece391-fa21/mp3-group-XX.git mp3
```

If you do have a ssh-key configured, clone your git repo in Z: drive by running the command:

```
git clone git@gitlab.engr.illinois.edu:ece391-fa21/mp3-group-XX.git mp3
```

Inside a newly created MP3 directory. You can add/delete files to/from the repository with `git add [file_list]` and `git rm [file_list]` respectively. Remember to `git pull` each time you sit down to work on the project and `git commit` and `git push` when you are done. Doing so will ensure that all members are working on the most current version of the sources. As a final note, it is bad practice to commit broken sources. You should make sure that your sources compile correctly before committing them to the repository. As with previous MPs, please do not modify the Makefile as we will be grading using the vanilla Makefile initially provided.

As you work on MP3 with your teammates, you may find it useful to create additional branches to avoid conflicts while editing source files. Remember to merge your changes back into the master branch by each checkpoint deadline, as this is the only branch we will use for grading.

3 Getting Started: Booting and GRUB

For this project, you will make use of GRUB (GRand Unified Bootloader) to boot your OS image file. GRUB implements the Multiboot specification, the details of which can be found online at <http://www.gnu.org/software/grub/manual/multiboot/multiboot.html>. You will need to read through at least the chunk of this documentation entitled "Boot information format" to understand the information that GRUB provides to your operating system upon bootup. Various boot parameters are stored in a `multiboot_info_t` data structure, whose address is passed in via `EBX` to the entry point of the OS image. The `multiboot.h` file contains this and other structure definitions that you should use when accessing the boot information.

GRUB drops you into protected mode, but with paging turned off and all the descriptor tables (GDT, LDT, IDT, TSS) in an undefined state. The first thing to do is set up a GDT, LDT, and TSS. The given code in `boot.S` does this for you, and then calls `entry()` in `kernel.c`, passing the address of the `multiboot_info_t` to it. GRUB has loaded the file system image at some place in physical memory, as a module; there is a section in the `multiboot_info_t` structure which describes how to access the module information, including physical addresses and size information. You will need to keep the base address of this file system module around, since many operations will need to interact with the file system. You may need to extract other boot information as well, such as the physical memory map provided to you by the BIOS (and also passed in as part of the boot information structure).

To get started, read the `INSTALL` file given in the MP distribution for instructions on booting the OS image file. Once you have the skeleton OS booting, you are ready to begin designing and implementing features. Necessary references for this project are the x86 ISA manuals, which can be found in the "Tools, References, and Links" section of the class website. *Volume 3: System Programming* details things like segmentation, virtual memory, interrupts and exceptions, and task support in all of their gory detail. The other volumes provide invaluable resources such as x86 instruction specifications. You will need to reference these manuals early and often; familiarize yourself with their contents before beginning. To debug your MP, you will take advantage of QEMU, which by now, you should be comfortable with.

One final note: GRUB depends on seeing the Multiboot header within the first 8 kB of the OS image file. This header is located in the `boot.S` file of your sources, and `boot.o` is passed as the first object file to the linker to ensure that it falls within the first 8 kB of the OS image. Don't move `boot.o` to a different position in the link order given in the Makefile, or GRUB may not be able to boot your OS image.

4 The Pieces

The materials provided to you will launch your machine into protected mode, set up the GDT and LDT as well as a TSS. A file system image with a shell, a few utilities, a single directory entry, and a real-time clock (RTC) device file will also be mapped into physical memory for you. We will also provide you with the tools necessary to develop your own test applications, including the compilation infrastructure used to produce the shell and utilities and the file system image itself (the `createfs`). For simplicity, we will stick to text-mode graphics, but your OS will in the end support operation of a user-level animation package from MP1. Some basic `printf` support is also provided to aid in debugging, but eventually you will need to write your own output support for the terminal.

Work plan:: Although we are not explicitly requiring that you tell us how you plan to split up the work for this project, we do expect that you will want to do so to allow independent progress by all team members. A suggested split of the work (after getting the devices initialized) is to have each person work on one of the subgoals of Checkpoint 2 with one person concentrating on how these functions will eventually integrate with system calls in Checkpoint 3 and 4, which connects all previous pieces using the system calls as glue. Checkpoint 5 also requires the group to work together on terminal drivers, RTC interrupts, and everything learned in Checkpoint 3 and 4.

Setting up a clean testing interface will also help with partitioning the work, since group members can finish and test parts of the project without other members having finished the other parts (yet).

5 Testing

For this project, we require you to demonstrate unit tests with adequate coverage of your source code.

As your operating system components are dependent on one another, you may find it useful to unit test each component individually to isolate any design or coding bugs.

We have provided you with a starter test suite in the `tests.c` file. As you add more components to your operating system, we encourage you to add corresponding tests that verify the functionality of each component at the interface level. Minimum test coverage for each checkpoint is detailed in the following sections.

You may launch tests individually from each of your source files, but all your tests should be defined in `tests.c`. This will be especially useful when passing in state variables from your modules to test functions. Remember to use the `RUN_TESTS` macro to exclude test launches when compiling your code for handin.

Keep in mind that passing all of your unit tests does not guarantee bug free code. However, the test suite provides a convenient means to run your tests frequently without having to re-debug older components as you add new functionality.

6 What to Hand in

6.1 Checkpoint 1: Processor Initialization

For the checkpoint, you must have the following accomplished:

6.1.1 Group Repository

You must have your code in the shared group repository, and each group member should be able to demonstrate that he/she can read and change the source code in the repository.

6.1.2 Load the GDT

You learned about the global descriptor table (GDT) in class. Linux creates four segments in this table: Kernel Code Segment, Kernel Data Segment, User Code Segment, and User Data Segment. In `x86_desc.S`, starting at line 38, we have created an empty GDT for you.

Write code that makes an emulated Intel IA-32 processor use this GDT. We have marked a location in the code (`boot.S` line 27) at which you will need to place this initialization code for the GDT to ensure that you follow the correct boot sequence. You will need to look through the ISA Reference Manual for information about how to write this code (<https://courses.engr.illinois.edu/ece391/secure/references/IA32-ref-manual-vol-3.pdf>).

6.1.3 Initialize the IDT

Your IDT must contain entries for exceptions, a few interrupts, and system calls. Consult the x86 ISA manuals for the descriptor formats and please see Appendix D for more information. The exception handler(s) should use the printing support to report errors when an exception occurs in the kernel, and should squash any user-level program that produces an exception, returning control to the shell (the shell should not cause an exception in a working OS)—see System Calls (Appendix B) for further details. You will also need to handle interrupts for the keyboard and the RTC. Finally, you will need to use entry `0x80` for system calls, as described below.

6.1.4 Initialize the Devices

Adapt the initialization code from Linux to initialize the PIC, the keyboard, and the RTC. Set up a general-purpose infrastructure similar to what is done in the Linux kernel. You need to handle the keyboard and RTC interrupts, but you also need to make sure that these devices are initialized before taking interrupts. We suggest that you first mask out all interrupts on the PIC, then initialize the PIC, initialize the devices, and, as part of each device's initialization, enable its associated interrupt on the PIC. The handler addresses should be installed dynamically/indirectly via a data structure used by the default handlers (as in Linux). You may also want to review the RTC data sheet linked from the class web page.

For the checkpoint, your OS must execute the `test_interrupts` handler (provided in `lib.c`) when an RTC interrupt occurs. When a keyboard interrupt occurs, you must echo the correct characters to the screen. These simple tests will determine if you have the IDT entries set up correctly, the PIC enabled, and the devices initialized and able to generate interrupts.

6.1.5 Initialize Paging

As preparation for the next steps in the MP you must have paging enabled and working. You will be creating a page directory and a page table with valid page directory entries (PDEs) and page table entries (PTEs). More information about this process appears in Appendix C and in the Intel ISA manual linked from the class web page.

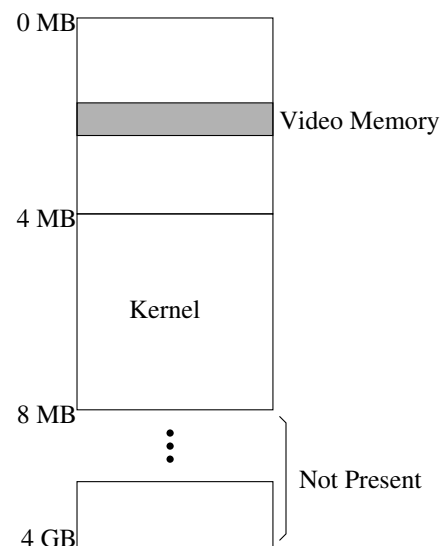
The image to the right shows how virtual and physical memory are laid out for this checkpoint. To keep things simple, the kernel and video memory will be at the same location in virtual memory as they are in physical memory. Your kernel code is already loaded at 4 MB for you, so you need only map virtual memory 4-8 MB to physical memory at 4-8 MB. This kernel page should be a single 4 MB page, whereas the first 4 MB of memory should be broken down into 4 kB pages. In addition to 8MB to 4GB being marked not present, you should also set any unused pages to not present as well. In this layout everything in the first 4MB, that isn't the page for video memory, should be marked not present.

Make sure that you align your pages (page directory and page tables) on 4 kB boundaries. To align things in x86:

```
.align BYTES_TO_ALIGN_TO
label:
(whatever you want aligned)
```

To align things in C:

```
int some_variable __attribute__((aligned (BYTES_TO_ALIGN_TO)));
```



6.1.6 Troubleshooting/Debugging

See Appendix G for more information about debugging and common issues.

6.1.7 Test Coverage

At minimum, your tests should cover:

- Values contained in the IDT array - An example has been provided in `tests.c`
- Receiving an RTC interrupt
- Interpreting various scancodes in the keyboard handler
- Values contained in your paging structures
- Dereferencing different address ranges with paging turned on
- Checking bad or garbage input and return values for any function you write

This list is subject to change. Keep an eye out for the rubric. You are encouraged to add more unit tests than those specified above.

6.1.8 Checkpoint 1 Handin

For all handins, we expect you to write your own unit tests and to use those to demonstrate functionality. For this handin, you should write your own “blue screen” of death for each of the exceptions. At a minimum, this screen must identify the exception taken. You may also want to read about how exceptions are handled in the Intel ISA and print more useful information, such as the memory address reference that caused a page fault. This information will be of use to you later for debugging. We expect you to be able to boot your operating system with paging turned on and to enter a `halt` loop or a `while (1) loop`. Then we expect you to boot your operating system and explicitly dereference `NULL` to demonstrate your “blue screen” identifying the resulting page fault. We also expect you to be able to press a key and demonstrate that your operating system reaches the `test_interrupts` function in on RTC interrupts, but you do **not** need to write a full interrupt handler for RTC yet, merely show that you can receive interrupts. Finally, we expect your keyboard interrupt handler to echo the correct character to the screen, although where each character appears on the screen doesn’t matter. As with the RTC, you need **not** write a full interrupt handler for the keyboard for this checkpoint. How you demonstrate these is up to you; do not rely on us to test it for you during handin. If you cannot prove functionality on your own, then we cannot give you points.

6.2 Checkpoint 2: Device Drivers

6.2.1 Create a Terminal Driver

When any printable characters are typed at the keyboard, they should be displayed to the screen. This includes handling all alphanumeric characters, symbols, shift and capslock, but you do not need to support the number pad. You will now need to keep track of the screen location for this purpose. You **do** need to support vertical scrolling (but not history) and **will** need to interpret CTRL-L (non-printable key) as meaning “clear the screen and put the cursor at the top” which will make your testing experience more pleasant. You **do** also need to support backspace and line-buffered input. The size of the buffer should be 128 characters for this checkpoint. For more details on how the terminal read and write functions should work, and how to handle the buffer filling up, please see Appendix B.

Keep in mind that you will also want to have an external interface to support delivery of external data to the terminal output. In particular, `write` system calls to the terminal should integrate cleanly with keyboard input. The `hello` program in the file system will eventually help to test the basics, but for now its source code will show you how user programs will pass parameters to your terminal. Moving forward, you should be reading through the other given source code to see how programs will consume its arguments and check (or not check, which then becomes your responsibility to check) for bad inputs.

6.2.2 Parse the Read-only File System

You will need to support operations on the file system image provided to you, including opening and reading from files, opening and reading the directory (there’s only one—the structure is flat), and copying program images into contiguous physical memory from the randomly ordered 4 kB “disk” blocks that constitute their images in the file system. The source code for our `ls` program will show you how reading directories is expected to work. Also see Appendix A for an overview of the file system as well as Appendix B for how each function should work.

6.2.3 The Real-Time Clock Driver

You will need to write the `open`, `read`, `write`, and `close` functions for the real-time clock (RTC) and demonstrate that you can change the clock frequency. You will need to do some research on how the RTC works and what the device driver needs to do to communicate with it. Virtualizing the RTC is not required, but does make testing easier when you run multiple programs with the RTC open. Again, see Appendix B for how each function should work.

6.2.4 Test Coverage

At minimum, your tests should cover:

- Reading in user input from the terminal/keyboard (don’t forget about the max buffer size!)
- Writing different sized strings to the terminal (what happens if you tell it to write a number of bytes that doesn’t match the buffer size?)
- Printing out a list of all files in the file system (their corresponding file size would be nice to see as well as a good sanity check for the next bullet point here)
- Printing out the contents of various different files (text, executables, small, large, etc.). For executable files, you may see garbage values (unless you are not human...). This is OK, but you should most definitely see a few patterns, namely one at the beginning of the file and one at the end of the file
- Changing RTC frequencies, receiving RTC interrupts at each possible frequency
- Checking bad or garbage input and return values for any function you write

This list is subject to change. Keep an eye out for the rubric. You are encouraged to add more unit tests than those specified above. Take a look at the program source code in the `syscalls/` folder for inspiration on how to write your tests.

6.2.5 Checkpoint 2 Handin

As before, we expect you to write your own test cases. For this handin, you will need to demonstrate that your `open`, `read`, and `write` functions for the three device drivers work correctly. This functionality is independent of how your operating system may use the devices, but it is a good idea for you to start thinking about how you want to interface these functions with the corresponding system calls (see Appendix B).

You will need to show that when a key is pressed, the keyboard driver stores the corresponding letter in a buffer and that by explicitly calling the keyboard `read` function you can receive the correct letters and print them out on the screen. Similarly, you will need to demonstrate that you can change the rate of the RTC clock using the `write` function and that the `read` function returns after an interrupt has occurred. A good way of doing so is by having the RTC interrupt handler increment a counter and write to a specific spot on the screen.

Finally, you will need to demonstrate that you can read from the read-only file system. A good check for this test is to use the `xxd` command, which prints a hexadecimal dump, to compare the output of your file system code with the bytes stored in the file system image. Please note that you should run `xxd` on **filesys.img** and not the individual executables; as your code reads from the filesystem image, its output might not directly match that of `xxd` run on the individual executables. For the handin, you must have some test/wrapper code that given a filename, a buffer, and a buffer length, will read data from the given file into the buffer. This can be part of your test suite.

6.3 Checkpoint 3: Starting System Calls and Tasks

6.3.1 Support System Calls

Eventually, all ten system calls must be supported via a common IDT entry, so you will have to set up some generic assembly linkage along the lines of that used in Linux, including syscall value checking, register save and restore, and a jump table to C functions that implement the system calls themselves. The details of each call are provided in Appendix B.

For this checkpoint, you only need to support the system calls necessary to run `testprint` from shell: `execute`, `halt`, and `open/close/read/write` for the terminal and filesystem.

6.3.2 Tasks

Programs execute to completion, so you need not write a scheduler or deal with the timer chip yet, but you do need to be able to squash programs if they generate exceptions, returning control to the shell in such cases.

As in Linux, the tasks will share common mappings for kernel pages, in this case a single, global 4 MB page. Unlike Linux, we will provide you with set physical addresses for the images of the two tasks, and will stipulate that they require no more than 4 MB each, so you need only allocate a single page for each task's user-level memory. See Appendix C for more details.

6.3.3 Support A Loader

Extend the code for your file system driver to copy a program image from the randomly ordered 4 kB “disk” blocks constituting the image in the file system into contiguous physical memory.

This process is normally performed by a program loader in cooperation with the OS, but in your case will be performed completely within the kernel, since the file system code and control of the memory is internal.

In addition, you will need to set up the stack properly and then return into user-level, since privilege level 0 cannot call down into privilege level 3, and your user-level code must execute at the lower privilege level.

6.3.4 Executing User-level Code

Kernel code executes at privilege level 0, while user-level code must execute at privilege level 3. The x86 processor does not allow a simple function call from privilege level 0 code to privilege level 3, so you must use an x86-specific convention to accomplish this privilege switch.

The convention to use is the `IRET` instruction. Read the ISA reference manual for the details of this instruction. You must set up the correct values for the user-level `EIP`, `CS`, `EFLAGS`, `ESP`, and `SS` registers on the kernel-mode stack, and then execute an `IRET` instruction. The processor will pop the values off the stack into those registers, and by doing this, will perform a privilege switch into the privilege level specified by the low 2 bits of the `CS` register. The values for the `CS` and `SS` registers must point to the correct entries in the Global Descriptor Table that correspond to the user-mode code and stack segments, respectively. The `EIP` you need to jump to is the entry point from bytes 24-27 of the executable that you have just loaded. Finally, you need to set up a user-level stack for the process. For simplicity, you may simply set the stack pointer to the bottom of the 4 MB page already holding the executable image. Two final bits: the `DS` register must be set to point to the correct entry in the GDT for the user mode data segment (`USER_DS`) before you execute the `IRET` instruction (conversely, when an entry into the kernel happens, for example, through a system call, exception, or interrupt, you should set `DS` to point to the `KERNEL_DS` segment). Finally, you will need to modify the TSS; this is explained in Appendix E.

6.3.5 Process Control Block

The next piece to support tasks in your operating system is per-task data structures, for example, the process control block (PCB). One bit of per-task state that needs to be saved is the file array, described earlier; another is the signal information. You may need to store some other things in the process control block; you must figure the rest out on your own. The final bit of per-task state that needs to be allocated is a kernel stack for each user-level program. Since you only need to support two tasks, you may simply place the first task's kernel stack at the bottom of the 4 MB kernel page that you have already allocated. The second task's stack can then go 8 kB above it. This way, both tasks will have 8 kB kernel stacks to use when inside the kernel. Each process's PCB should be stored at the top of its 8 kB stack, and the stack should grow towards them. Since you've put both stacks inside the 4 MB page, there is no need to "allocate" memory for the process control block. To get at each process's PCB, you need only AND the process's `ESP` register with an appropriate bit mask to reach the top of its 8 kB kernel stack, which is the start of its PCB. Finally, when a new task is started with the `execute` system call, you'll need to store the parent task's PCB pointer in the child task's PCB so that when the child program calls `halt`, you are able to return control to the parent task.

6.3.6 Test Coverage

At minimum, your tests should cover:

- Checking bad or garbage input and return values for any function you write
- Running the shell and testprint programs
- Halting the shell and testprint programs
- Using the read/write system calls to read/write to the terminal
- Checking the multiple steps of the `execute` system call
- Checking the open/close/read/write system calls properly created, initialized, used, and cleaned up the file descriptor array

This list is subject to change. Keep an eye out for the rubric. You are encouraged to add more unit tests than those specified above. Take a look at the program source code in the `syscalls/` folder for inspiration on how to write your tests.

6.4 Checkpoint 4: Completing System Calls and Tasks

For this handin, we expect that you have all of the system calls working and that all of the programs we have provided to you will execute without problems. You are also expected to handle the multiple "shell" case where you `execute` a shell from the first shell. Just for this checkpoint, you can assume a maximum of two programs. Further programs should not be allowed to run. You can exit shells by typing "exit", and return to the previous shell. As mentioned previously, you should be looking through the given source code to ensure you understand how programs pass parameters and check (or don't check) bad parameters.

6.4.1 Test Coverage

At minimum, your tests should cover:

- Verifying values in your vidmap paging structure
- Running more than one program

- Checking that all user programs work (make sure you know the behavior of those programs when given different arguments)
- Checking bad or garbage input and return values for any function you write

This list is subject to change. Keep an eye out for the rubric. You are encouraged to add more unit tests than those specified above. Take a look at the program source code in the `syscalls/` folder for inspiration on how to write your tests.

6.5 Checkpoint 5: Scheduling

For the final due date, the following functionality must be implemented:

6.5.1 Multiple Terminals and Active Tasks

As you may already know, it is possible to switch between different terminals in Linux using the **ALT+Function-Key** combination. You will need to add a similar feature by running several instances of the `shell` executable. You must support three terminals, each associated with a different instance of `shell`. As an example, pressing **ALT+F2** while in the first terminal must switch to the active task of the second terminal. Further, you must support up to six processes in total. For example, each terminal running shell running another program. For the other extreme, have 2 terminals running 1 shell and have 1 terminal running 4 programs (a program on top of shell, on top of shell, *etc.*).

In order to support the notion of a terminal, you must have a separate input buffer associated with each terminal. In addition, each terminal should save the current text screen and cursor position in order to be able to return to the correct state. Switching between terminals is equivalent to switching between the associated active tasks of the terminals. Finally, your keyboard driver must intercept **ALT+Function-Key** combinations and perform terminal switches.

Lastly, keep in mind that even though a process can be interrupted in either user mode or in kernel mode (while waiting in a system call). After the interrupt, the processor will be in kernel mode, but the data saved onto the stack depends on the state before the interrupt. Each process should have its own kernel stack, but be careful not to implicitly assume either type of transition.

6.5.2 Scheduling

Until this point, task switching has been done by either executing a new task or by halting an existing one and returning to the parent task. By adding a scheduler, your OS will actively preempt a task in order to switch to the next one. Your OS scheduler should keep track of all tasks and schedule a timer interrupt every 10 to 50 milliseconds in order to switch to the next task in a round-robin fashion.

When adding a scheduler, it is important to keep in mind that tasks running in an inactive terminal should not write to the screen. In order to enforce this rule, a remapping of virtual memory needs to be done for each task. Specifically, the page tables of a task need to be updated to have a task write to non-display memory rather than display memory when the task is not the active one (see the previous section).

If the task belongs to the active terminal, the video memory virtual address should be mapped to the physical video memory address. Otherwise, these virtual addresses must map into different physical pages allocated as a backing store for the task's screen data. These backing pages are then written whenever the task calls `write` on the standard output. Eventually, when the user makes the task's terminal active again, your OS must copy the screen data from the backing store into the video memory and re-map the virtual addresses to point to the video memory's physical address.

6.5.3 Test Coverage

At minimum, your tests should cover:

- Verifying that all the previous functionalities still work for multiple terminals
- Verifying that all the previous functionalities still work for scheduling
- Verifying mappings in your video paging structures
- Testing your data structures that track scheduled processes
- Checking bad or garbage input and return values for any function you write

This list is subject to change. Keep an eye out for the rubric. You are encouraged to add more unit tests than those specified above.

6.5.4 Checkpoint 5 Handin

For this final handin, we expect you to demonstrate that multiple terminals work by switching between active terminals. We will execute a program on one screen, switch to another screen, start another program, and then expect to be able to switch back and forth to see the programs running. For scheduling, we expect that programs running in the background (on an inactive terminal) will make progress. For example, we expect a fish program running on such a terminal to continue despite not actually being displayed on the screen.

6.6 Extra Credit

We will hold a design competition at the end of the semester. In order to be eligible for this competition, your operating system **must implement all required functionality correctly**. The competition will then be decided based on the amount and difficulty of additional functionality that your team has incorporated into your operating system.

You may also be able to earn some extra credit (or make up for other lost points) by choosing to include some of the features in this section. Extra credit earned in this way will be limited to 10 points of the baseline grade.

In order to be eligible for extra credit, your operating system **must implement all required functionality correctly**. Extra credit is due by the design competition, however you may turn in extra credit with your final checkpoint handin. Make sure your extra credit does not break anything or make your OS unstable.

6.6.1 Signals

Add support for delivery of five different signals to a task. Appendix F details the specifications and implementation details to make this work.

6.6.2 Dynamic Memory Allocation

Create a dynamic memory allocator (such as `malloc`). This can be done by simply keeping track of where free pages are using some method, then creating a `malloc` system call that adds a new page to the program's page table or page directory.

Remember that implementing dynamic memory will not get you any extra credit if you don't have a way of demoing the functionality. For example, writing a short user program or some kind of kernel-level functionality.

6.6.3 Other Ideas

Go wild, find something interesting to add to your operating system and explain it. We will consider the difficulty of any addition when determining whether and how much extra credit is merited. For example, don't expect a huge grade increase for adding color support for text.

7 Grading

Your final MP score is a baseline score that will be adjusted for teamwork and for individual effort. Note that the “correct” behavior of certain routines includes interfaces that allow one to prevent buffer overflows (that is, the interfaces do not leave the size of a buffer as an unknown) and other such behavior. While the TAs will probably not have time to sift through all of your code in detail, they will read parts of it and look at your overall design. The rough breakdown of how points will be distributed will be periodically released on the website prior to checkpoint deadlines.

A Note on Teamwork

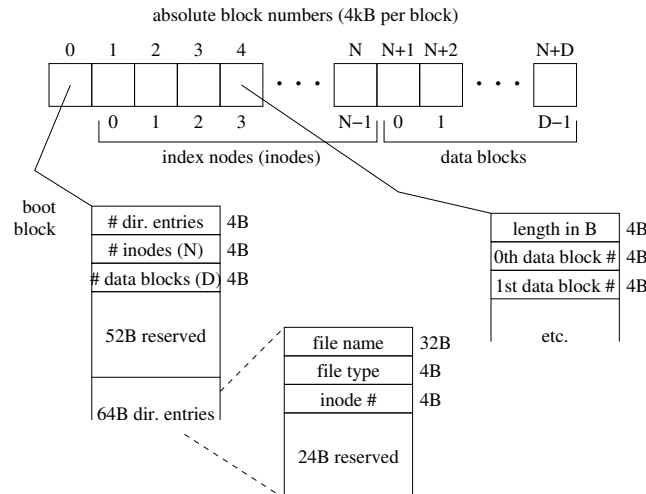
Teamwork is an important part of this class and will be used in the grading of this MP. We expect that you will work to operate effectively as a team, leveraging each member’s strengths and making sure that everyone understands how the system operates to the extent that they can explain it. In the final demo, for example, we will ask each of the team members questions and expect that they will be able to answer at a reasonable level **without** referring to another team member. **Failure to operate as a team will significantly reduce your overall grade for this MP.**

We will also ask each of you to apportion credit for the overall MP to each of the other team members (not including yourself) in order to gauge how contributions were balanced amongst your team. This information will be used to adjust your final score for the MP. Teams that operate smoothly together are free to opt for simply marking the form as equal, which is fine, but the information that you provide about relative effort is treated as confidential. Note that you cannot affect your own grade through any choice of credit assignment, only those of your teammates (and vice-versa).

8 Appendix A: The File System

8.1 File System Utilities

The figure below shows the structure and contents of the file system. The file system memory is divided into 4 kB blocks. The first block is called the boot block, and holds both file system statistics and the directory entries. Both the statistics and each directory entry occupy 64B, so the file system can hold up to 63 files. The first directory entry always refers to the directory itself, and is named “.”, so it can really hold only 62 files.



Each directory entry gives a name (up to 32 characters, zero-padded, but *not necessarily including a terminal EOS or 0-byte*), a file type, and an index node number for the file. File types are 0 for a file giving user-level access to the real-time clock (RTC), 1 for the directory, and 2 for a regular file. The index node number is only meaningful for regular files and should be ignored for the RTC and directory types.

Each regular file is described by an index node that specifies the file’s size in bytes and the data blocks that make up the file. Each block contains 4 kB; only those blocks necessary to contain the specified size need be valid, so be careful not to read and make use of block numbers that lie beyond those necessary to contain the file data.

```
int32_t read_dentry_by_name (const uint8_t* fname, dentry_t* dentry);
int32_t read_dentry_by_index (uint32_t index, dentry_t* dentry);
int32_t read_data (uint32_t inode, uint32_t offset, uint8_t* buf, uint32_t length);
```

The three routines provided by the file system module return -1 on failure, indicating a non-existent file or invalid index in the case of the first two calls, or an invalid inode number in the case of the last routine. Note that the directory entries are indexed starting with 0. Also note that the `read_data` call can only check that the given inode is within the valid range. It does not check that the inode actually corresponds to a file (not all inodes are used). However, if a bad data block number is found within the file bounds of the given inode, the function should also return -1.

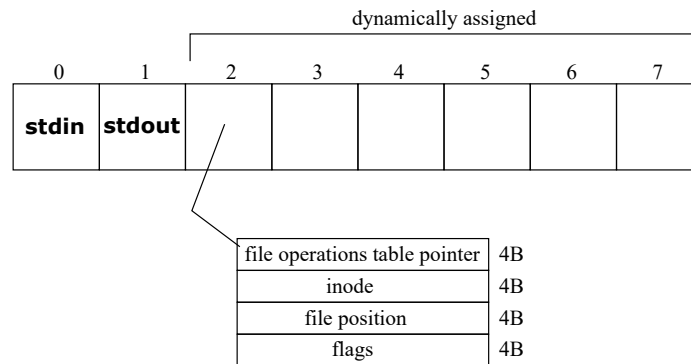
When successful, the first two calls fill in the `dentry_t` block passed as their second argument with the file name, file type, and inode number for the file, then return 0. The last routine works much like the `read` system call, reading up to `length` bytes starting from position `offset` in the file with inode number `inode` and returning the number of bytes read and placed in the buffer. A return value of 0 thus indicates that the end of the file has been reached.

8.2 File System Abstractions

Each task can have up to 8 open files. These open files are represented with a file array, stored in the process control block (PCB). The integer index into this array is called a **file descriptor**, and this integer is how user-level programs identify the open file.

This array should store a structure containing:

1. The file operations jump table associated with the correct file type. This jump table should contain entries for `open`, `read`, `write`, and `close` to perform type-specific actions for each operation. `open` is used for performing type-specific initialization. For example, if we just `open`'d the RTC, the jump table pointer in this structure should store the RTC's file operations table.
2. The inode number for this file. This is only valid for data files, and should be 0 for directories and the RTC device file.
3. A "file position" member that keeps track of where the user is currently reading from in the file. Every `read` system call should update this member.
4. A "flags" member for, among other things, marking this file descriptor as "in-use."



When a process is started, the kernel should automatically open `stdin` and `stdout`, which correspond to file descriptors 0 and 1, respectively. `stdin` is a read-only file which corresponds to keyboard input. `stdout` is a write-only file corresponding to terminal output. "Opening" these files consists of storing appropriate jump tables in these two locations in the file array, and marking the files as in-use. For the remaining six file descriptors available, an entry in the file array is dynamically associated with the file being `open`'d whenever the `open` system call is made (return -1 if the array is full).

9 Appendix B: The System Calls

You must support ten system calls, numbered 1 through 10. As with Linux, they are invoked using `int $0x80`, and use a similar calling convention. In particular, the call number is placed in EAX, the first argument in EBX, then ECX, and finally EDX. No call uses more than three arguments, although you should protect all of the registers from modification by the system call to avoid leaking information to the user programs. The return value is placed in EAX if the call returns (not all do); a value of -1 indicates an error, while others indicate some form of success.

Prototypes appear below. Unless otherwise specified, successful calls should return 0, and failed calls should return -1.

```
1. int32_t halt (uint8_t status);
2. int32_t execute (const uint8_t* command);
3. int32_t read (int32_t fd, void* buf, int32_t nbytes);
4. int32_t write (int32_t fd, const void* buf, int32_t nbytes);
5. int32_t open (const uint8_t* filename);
6. int32_t close (int32_t fd);
7. int32_t getargs (uint8_t* buf, int32_t nbytes);
8. int32_t vidmap (uint8_t** screen_start);
9. int32_t set_handler (int32_t signum, void* handler_address);1
10. int32_t sigreturn (void);2
```

The `execute` system call attempts to load and execute a new program, handing off the processor to the new program until it terminates. The command is a space-separated sequence of words. The first word is the file name of the program to be executed, and the rest of the command—stripped of leading spaces—should be provided to the new program on request via the `getargs` system call. The `execute` call returns -1 if the command cannot be executed, for example, if the program does not exist or the filename specified is not an executable, 256 if the program dies by an exception, or a value in the range 0 to 255 if the program executes a `halt` system call, in which case the value returned is that given by the program's call to `halt`.

The `halt` system call terminates a process, returning the specified value to its parent process. The system call handler itself is responsible for expanding the 8-bit argument from BL into the 32-bit return value to the parent program's `execute` system call. Be careful not to return all 32 bits from EBX. This call should never return to the caller.

The `read` system call reads data from the keyboard, a file, device (RTC), or directory. This call returns the number of bytes read. If the initial file position is at or beyond the end of file, 0 shall be returned (for normal files and the directory). In the case of the keyboard, read should return data from one line that has been terminated by pressing Enter, or as much as fits in the buffer from one such line. The line returned should include the line feed character. In the case of a file, data should be read to the end of the file or the end of the buffer provided, whichever occurs sooner. In the case of reads to the directory, only the filename should be provided (as much as fits, or all 32 bytes), and subsequent reads should read from successive directory entries until the last is reached, at which point `read` should repeatedly return 0. For the real-time clock (RTC), this call should always return 0, but only after an interrupt has occurred (set a flag and wait until the interrupt handler clears it, then return 0). You should use a jump table referenced by the task's file array to call from a generic handler for this call into a file-type-specific function. This jump table should be inserted into the file array on the `open` system call (see below).

The `write` system call writes data to the terminal or to a device (RTC). In the case of the terminal, all data should be displayed to the screen immediately. In the case of the RTC, the system call should always accept only a 4-byte integer specifying the interrupt rate in Hz, and should set the rate of periodic interrupts accordingly. Writes to regular files should always return -1 to indicate failure since the file system is read-only. The call returns the number of bytes written, or -1 on failure.

The RTC device itself can only generate interrupts at a rate that is a power of 2 (do a parameter check), and only up to 8192 Hz. Your kernel should limit this further to 1024 Hz — an operating system shouldn't allow user

¹Extra credit.

²Also for extra credit.

space programs to generate more than 1024 interrupts per second by default. Look at `drivers/char/rtc.c`, `include/linux/mc146818rtc.h` and possibly other associated header files for the macros and port numbers for interfacing with the RTC device.

Also, see the datasheet at

<https://courses.engr.illinois.edu/ece391/secure/references/mc146818.pdf> (old)

and

<https://courses.engr.illinois.edu/ece391/secure/references/ds12887.pdf> (new)

for details on registers and other parameters of the hardware.

Note that you should be using the RTC's Periodic Interrupt function to generate interrupts at a programmable rate. The RTC interrupt rate should be set to a default value of 2 Hz (2 interrupts per second) when the RTC device is opened. For simplicity, RTC interrupts should remain on at all times.

The `open` system call provides access to the file system. The call should find the directory entry corresponding to the named file, allocate an unused file descriptor, and set up any data necessary to handle the given type of file (directory, RTC device, or regular file). If the named file does not exist or no descriptors are free, the call returns -1.

The `close` system call closes the specified file descriptor and makes it available for return from later calls to `open`. You should not allow the user to close the default descriptors (0 for input and 1 for output). Trying to close an invalid descriptor should result in a return value of -1; successful closes should return 0.

The `getargs` call reads the program's command line arguments into a user-level buffer. Obviously, these arguments must be stored as part of the task data when a new program is loaded. Here they are merely copied into user space. If there are no arguments, or if the arguments and a terminal NULL (0-byte) do not fit in the buffer, simply return -1. The shell does not request arguments, but you should probably still initialize the shell task's argument data to the empty string.

The `vidmap` call maps the text-mode video memory into user space at a pre-set virtual address. Although the address returned is always the same (see the memory map section later in this handout), it should be written into the memory location provided by the caller (which must be checked for validity). If the location is invalid, the call should return -1. To avoid adding kernel-side exception handling for this sort of check, you can simply check whether the address falls within the address range covered by the single user-level page. Note that the video memory will require you to add another page mapping for the program, in this case a 4 kB page. It is **not** ok to simply change the permissions of the video page located < 4MB and pass that address.

The `set_handler` and `sigreturn` calls are related to signal handling and are discussed in the section **Signals** below. Even if your operating system does not support signals, you must support these system calls; in such a case, however, you may immediately return failure from these calls.

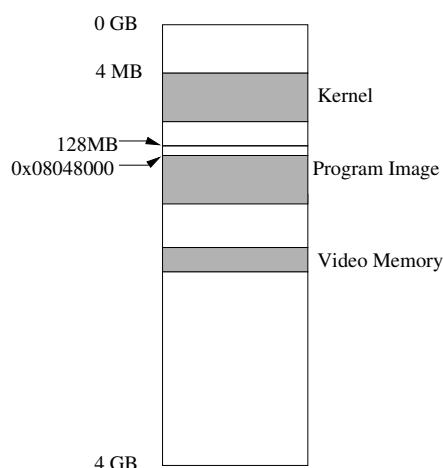
Note that some system calls need to synchronize with interrupt handlers. For example, the `read` system call made on the RTC device should wait until the next RTC interrupt has occurred before it returns. Use simple volatile flag variables to do this synchronization (e.g., something like `int rtc_interrupt_occurred;`) when possible (try something more complicated only after everything works!), and small critical sections with `cli/sti`. For example, writing to the RTC should block interrupts to interact with the device. Writing to the terminal also probably needs to block interrupts, if only briefly, to update screen data when printing (keyboard input is also printed to the screen from the interrupt handler).

10 Appendix C: Memory Map and Task Specification

When processing the `execute` system call, your kernel must create a virtual address space for the new process. This will involve setting up a new Page Directory with entries corresponding to the figure shown on the right.

The virtual memory map for each task is shown in the figure. The kernel is loaded at physical address `0x400000` (4 MB), and also mapped at virtual address 4 MB. A global page directory entry with its Supervisor bit set should be set up to map the kernel to virtual address `0x400000` (4 MB). This ensures that the kernel, which is linked to run with its starting address at 4 MB, will continue to work even after paging is turned on.

To make physical memory management easy, you may assume there is at least 16 MB of physical memory on the system. Then, use the following (static) strategy: the first user-level program (the shell) should be loaded at physical 8 MB, and the second user-level program, when it is executed by the shell, should be loaded at physical 12 MB. The program image itself is linked to execute at virtual address `0x08048000`. The way to get this working is to set up a single 4 MB page directory entry that maps virtual address `0x08000000` (128 MB) to the right physical memory address (either 8 MB or 12 MB). Then, the program image must be copied to the correct offset (`0x00048000`) within that page.



Both the kernel mapping and the user-level program mapping are critical; memory references in neither the kernel nor the program will not work correctly unless they are mapped at these exact addresses.

The layout of executable files in the file system is simple: the entire file stored in the file system is the image of the program to be executed. In this file, a header that occupies the first 40 bytes gives information for loading and starting the program. The first 4 bytes of the file represent a “magic number” that identifies the file as an executable. These bytes are, respectively, 0: `0x7f`; 1: `0x45`; 2: `0x4c`; 3: `0x46`. If the magic number is not present, the `execute` system call should fail. The other important bit of information that you need to execute programs is the entry point into the program, i.e., the virtual address of the first instruction that should be executed. This information is stored as a 4-byte unsigned integer in bytes 24-27 of the executable, and the value of it falls somewhere near `0x08048000` for all programs we have provided to you. When processing the `execute` system call, your code should make a note of the entry point, and then copy the entire file to memory starting at virtual address `0x08048000`. It then must jump to the entry point of the program to begin execution. The details of how to jump to this entry point are explained in the next section.

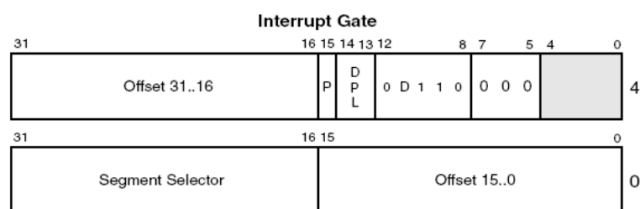
11 Appendix D: System Calls, Exceptions, and Interrupts

Recall that when a hardware interrupt is asserted or a hardware exception is detected, a specific number is associated with the exception or interrupt to differentiate between different types of exceptions, or different hardware devices (for example, differentiating between the keyboard interrupt, the network card interrupt, and a divide-by-zero exception). This number is used to index a table, called the **Interrupt Descriptor Table**, or IDT. The format of an IDT entry is shown in the figure below, and the details of it are found in the Intel architecture manuals (see the “Getting Started” section for more info on references). Each IDT entry contains, among other things, a pointer to the corresponding interrupt handler function to be run when that interrupt is received. When an exception or hardware interrupt is detected, the processor switches into privilege level 0 (kernel mode), saves some, but not all of the processor registers on the kernel stack (see Appendix E for more details), and jumps to the function address specified in the entry. Now, kernel code, specifically the interrupt handler for the correct interrupt number, is now executing.

For system calls, you will use a similar mechanism. A user-level program will execute a `int $0x80` instruction. The `int` instruction functions similar to an exception. It specifies that the processor should use entry `0x80` in the IDT as the handler when this instruction is executed. The same privilege-level switching, stack switching, *etc.*, are all performed, so after this instruction is run, kernel code will be executing. You must set up a “system call handler” IDT entry at index `0x80`, as well as a function that will be run for all system calls. This function can then differentiate different system calls based on the parameter passed in `EAX`.

For this to work properly, you must pay attention to a few details of the x86 architecture and protection scheme when setting up the IDT. The IDT will contain entries for exception handlers, hardware interrupt handlers, and the system call handler. Each entry in the IDT has a Descriptor Privilege Level (DPL) that specifies the privilege level needed to use that descriptor. Hardware interrupt handlers and exception handlers should have their DPL set to 0 to prevent user-

level applications from calling into these routines with the `int` instruction. The system call handler should have its DPL set to 3 so that it *is* accessible from user space via the `int` instruction. Finally, each IDT entry also contains a segment selector field that specifies a code segment in the GDT, and you should set this field to be the kernel’s code segment descriptor. When the x86 sees that a new CS is specified, it will perform a privilege switch, and the handler for the IDT entry will run in the new privilege level. This way, the system call interface is accessible to user space but the code executes in the kernel.



12 Appendix E: Stack Switching and the TSS

The last detail of user space to kernel transitions on system calls, interrupts, or exceptions is stack switching. The stack switch is taken care of by the x86 hardware. The x86 processor supports the notion of a task; this hardware support is encapsulated in a **Task State Segment**, or TSS. You will not use the full x86 hardware support for tasks in this project, but the x86 requires that you set up one TSS for, among other things, privilege level stack switching. The TSS in the given code is a placeholder. Read the Intel manuals for details on the fields in it; the important fields are `SS0` and `ESP0`. These fields contain the stack segment and stack pointer that the x86 will put into `SS` and `ESP` when performing a privilege switch from privilege level 3 to privilege level 0 (for example, when a user-level program makes a system call, or when a hardware interrupt occurs while a user-level program is executing). These fields must be set to point to the kernel’s stack segment and the process’s kernel-mode stack, respectively. Note that when you start a new process, just before you switch to that process and start executing its user-level code, you must alter the TSS entry to contain its new kernel-mode stack pointer. This way, when a privilege switch is needed, the correct stack will be set up by the x86 processor.

13 Appendix F: Signals

For extra credit, your OS can provide an infrastructure for user-level signals, similar to Linux. The table details the signals that will be supported.

Signal name	Signal number	Default action
DIV_ZERO	0	Kill the task
SEGFAULT	1	Kill the task
INTERRUPT	2	Kill the task
ALARM	3	Ignore
USER1	4	Ignore

The `set_handler` system call changes the default action taken when a signal is received: the `signum` parameter specifies which signal's handler to change, and the `handler_address` points to a user-level function to be run when that signal is received. It returns 0 if the handler was successful set, and -1 on failure. If `handler_address` is NULL (zero), the kernel should reset the action taken to be the default action.

DIV_ZERO should be sent to a task when the x86 processor generates a divide-by-zero exception while executing user-level code. SEGFAULT should be sent when any other exception occurs, including any illegal instructions, illegal memory references, page faults, general protection faults, illegal opcodes, *etc.* The INTERRUPT signal should be sent when a CTRL+C is pressed on the keyboard. The ALARM signal should be sent to the currently-executing task (there is only one currently-executing task in this OS) every 10 seconds. This should be implemented by knowing at what rate the RTC's Periodic Interrupts occur, counting how many Periodic Interrupts have occurred, and sending an ALARM signal after 10 seconds have elapsed. Finally, USER1 is user-defined and can be used to implement any other signal of your choosing.

Signals should only be delivered to a task when returning to user space from the kernel, so you'll want to add some code in your return-to-user space linkage to check for pending signals. To support signal delivery, you should use a mechanism similar to what Linux uses:

1. Mask all other signals.
2. Set up the signal handler's stack frame. You'll need the current value of the user-level ESP register to find the user's current stack location. The signal handler stack frame goes directly above this on the stack.

The signal handler stack frame is shown in Figure 1. Setting up the signal handler stack frame involves: copying a return address and a signal number parameter to the user-level stack, copying the process's hardware context (see Figure 2) from the point when the program was interrupted for the signal, and copying a small amount of assembly linkage to the user-level stack that calls `sigreturn` when the signal handler is finished.

3. Finally, execute (in user space) the handler specified in the signal descriptor. No other information needs to be passed to the signal handler (no `siginfo_t` structure like the modern Linux signals).

When the user-level signal handler returns, it will use the return address you have copied on its stack, which will jump to the assembly linkage (also on the stack). This assembly linkage should make the `sigreturn` system call (using the standard `int $0x80` user-level system call calling convention).

The `sigreturn` system call should copy the hardware context that was on the user-level stack back onto the processor. To find the hardware context, you will need to know the user-level value of ESP (will be saved by your system call handler) as well as the exact setup of the user-level stack frame. To copy the hardware context back onto the processor, you will actually overwrite the kernel's copy of the process's hardware context that was saved on the *kernel* stack when it handled the `sigreturn` system call. In this way, when the `sigreturn` system call handler returns to user space, the hardware context will automatically be copied back onto the processor by your return-from-kernel code that you have already written. One thing to be careful of: you'll probably have system calls set up to return a value (in EAX) to user space. Be sure you don't clobber the user's EAX value from its hardware context with a bogus "return value" from `sigreturn` – have `sigreturn` return the hardware context's EAX value so that you won't have to special-case the return from `sigreturn`.

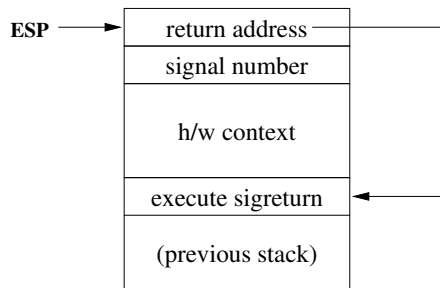


Figure 1: User-level signal handler stack frame.

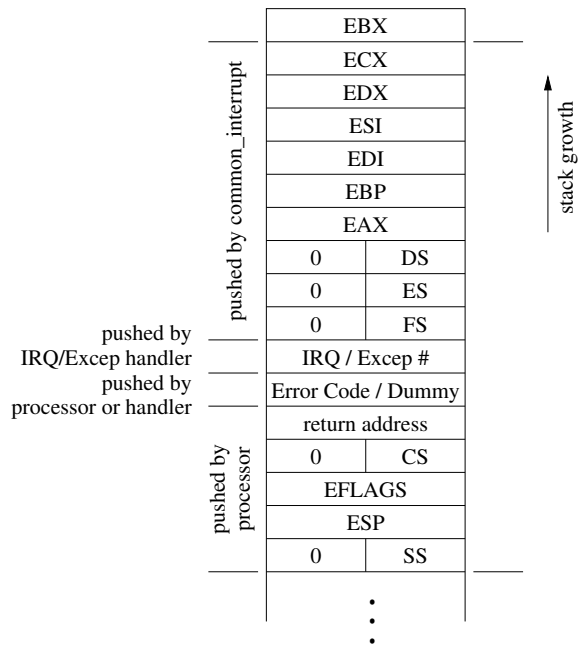


Figure 2: Hardware context structure.

Shown in Figure 2 is a slightly-modified version of the `struct ptregs` structure that Linux uses for its hardware context; this modified structure is what you should use in this MP. The “Error Code / Dummy” field has been added to the hardware context to simplify exception handling. For some exceptions, the processor pushes an error code onto the stack after XCS (for example, page faults do) whereas other exceptions do not (divide-by-zeros do not). For exceptions that do not push this error code, your exception handler should push a dummy value to take up the error code slot³. x86 interrupts never push the error code, so you must also push a dummy value in all of your interrupt handlers and the system call handler. You can find documentation about which exceptions push error codes (and much more about exceptions) in *Volume III: System Programming* of the Intel ISA manual on the Tools, References, and Links section of the website.

Finally, signal handling information should go in the process control block (PCB). You will need to keep track of pending signals, masked signals, and handler actions / addresses for each signal. Much information on Linux’s implementation of signals (which your implementation will closely match) can be found in *Understanding the Linux Kernel* chapter 10.

³Linux’s `struct ptregs` does not include this error code field; instead, the Linux exception handlers play tricks with the stack and registers to avoid adding an extra slot.

14 Appendix G: Troubleshooting

14.1 Debugging with QEMU

This section describes how to build your new OS which will be embedded, together with the `filesys.img`, in `mp3.img`.

Whenever a change is made to your kernel or file system, you need to `sudo` make a new kernel, which consequently prepares the `mp3.img` file which is used in the newly modified `test_debug.lnk` file.

Your new `test_debug.lnk` file should be modified to pass the new mp3 QEMU image. Change the target line to `"c:\qemu-1.5.0-win32-sdl\qemu-system-i386w.exe" -hda "<mp3 directory>\mp3.img" -m 256 -gdb tcp:127.0.0.1:1234 -S -name mp3`

You may also write the following to a `.bat` file to accomplish the same thing:

```
c:
cd "c:\qemu-1.5.0-win32-sdl\"
qemu-system-i386w.exe -hda <mp3 directory>\mp3.img -m 256 -gdb tcp:127.0.0.1:1234
-S -name mp3
```

where `<mp3 directory>` is likely to be: `z:\mp3\student-distrib`. In order to start debugging with GDB, run your `test_debug.lnk` file, then issue the following commands in the development machine:

```
cd <mp3_directory>/student-distrib
gdb bootimg
target remote 10.0.2.2:1234
```

GDB should now be started and connected to QEMU. From here, you can set up break points and everything else that you would normally do with GDB. Type `c` to continue execution instead of `r` since you connected to QEMU which is already running. When you do continue execution in GDB, GRUB will load first in QEMU. You need to hit enter, or wait 5 seconds, for your OS to load. QEMU is known to crash if your page tables are incorrectly setup. You might have to use the task manager in Windows to kill QEMU if this happens.

14.2 /mnt/tmpmp3 Compile Error

When compiling your kernel in MP3, you must first close the test machine. If you forget to close the test machine while compiling, or forget to run as root when you `make`, you will need to remove the old MP3 image before you can compile again. The commands below can be added to a script to remove the broken files and correct the issue (the last command should be run in `git-bash`).

```
sudo rm -rf /mnt/tmpmp3
rm bootimg
rm mp3.img
git checkout mp3.img
```

If you run the above steps and you still can't compile, the `mp3.img` in your repo is broken. You will need to revert back to an earlier version of `mp3.img` or you will need to grab a fresh `mp3.img` from the class directory under `mp3/student-distrib`.

14.3 Buffer I/O error on device loop0, logical block ##### lost page write due to I/O error on loop0

While this error may be caused by many issues, it's likely it was caused by an `mp3.img` corruption. Follow the above steps to attempt a fix.